LLM-Driven Unit Test Generation with a Dynamic Multi-Source Feedback Graph and Jev Typed Decision

Author Name
Affiliation placeholder – to be filled before submission
author@example.edu

Abstract

Large language models (LLMs) have become a dominant paradigm for unit test generation, yet most systems still rely on a *static*, coverage-first ordering of generation intents that ignores the heterogeneous signals produced while a test suite is being built. We present **DMF-Jev**, a training-free agent framework that couples a *Dynamic Multi-source Feedback Graph* (six live signals: coverage, mutation, compilation, execution, history, and code structure) with a *Jev typed decision* layer borrowed from the closed-choice decision literature (REFLEX). The decision layer routes each "next intent" through a frozen confidence band at $\tau = 0.5$: high-confidence choices are taken directly by the cheap Jev selector, while low-confidence choices fall back to the LLM. We specify the framework, the benchmark protocol (CodaMosa's 486-module suite and the TestEval 210-item set), the baselines (codamosa, coverup, telpa, and the Undermind agent strategy), and an explicit data post-processing pipeline. All quantitative results in this draft are computed from a seeded synthetic generator (DEMO) to exercise the full pipeline end to end; they are explicitly flagged as non-measurements, and the manuscript separates real from simulated data by construction.

1 Introduction

Unit test generation is a long-standing software-engineering problem that has been repeatedly reshaped by the arrival of stronger models. Classical search-based tools (EvoSuite [Fraser & Arcuri(2017)], Pynguin [Lukasczyk & Fraser(2022)]) optimize coverage with randomized search; the LLM wave (ChatUniTest [Chen et al.(2024)], TestPilot [Schäfer et al.(2023)], TestForge) replaced hand-crafted objectives with generation-validation loops driven by compilation and execution feedback. Despite these advances, a structural limitation persists: *the order in which test intents are pursued is still decided by a static, coverage-first heuristic* (the `order_intentions`/`top_k` family), which cannot exploit the rich, time-varying feedback a running agent observes.

We argue that the right abstraction is a *feedback graph*, not a ranked list. As a test-generation agent acts, it continuously produces coverage deltas, mutation-kill information, compilation outcomes, runtime errors, historical success patterns, and code-structure cues. These signals are correlated and task-dependent; a fixed priority ignores them.

Concretely, consider a module where search has stalled at 82% branch coverage with two uncovered branches left: one behind an exception guard that never fires in practice, one reachable only through a rarely used keyword argument. A static coverage-first scheduler keeps re-prioritizing both branches by uncovered-line count and burns LLM calls on the unreachable one. A feedback graph that folds in execution traces and code-structure cues can down-rank the first branch (no feasible path) and route the second to a targeted, argument-aware intent – and, because the choice

between "target branch A" and "target branch B" is a *closed choice* over a small typed vocabulary, a cheap selector with a confidence estimate can make it without invoking the LLM at all.

Our contribution is to (1) represent these signals as a dynamic multi-source feedback graph, (2) replace the static sorter with a *Jev typed decision* layer that makes a constrained, closed-choice selection per intent, and (3) gate that layer with a frozen confidence band so that the expensive LLM is invoked only when necessary. In summary:

- **Formulation.** We cast test-intent selection as closed-choice routing over a dynamic multi-source feedback graph, unifying six heterogeneous signals under one online-updated representation (§4).
- **Mechanism.** We instantiate the routing layer as a Jev typed decision with a frozen confidence band ($\tau = 0.5$), separating cheap high-confidence choices from LLM fallbacks.
- **Protocol.** We define an explicitly measured, boundary-declared evaluation protocol (benchmarks, baselines, metrics, post-processing) that separates real measurements from synthetic placeholders by construction (§6).
- **Artifact.** The full pipeline – data generation, statistics, figure building, and LaTeX assembly – is scripted and re-runnable end to end; every number in this manuscript can be regenerated from a single seed.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 derives the research questions. Section 4 specifies DMF-Jev. Sections 5–6 describe the experimental design and post-processing pipeline. Section 7 reports (demo) results, Section 8 discusses implications and limitations, and Section 9 concludes.

2 Related Work

Search-based and LLM-based generation. EvoSuite [Fraser & Arcuri(2017)] and Pynguin [Lukasczyk & Fra established whole-suite and assertion-focused search. The LLM era introduced generation-validation pipelines (ChatUniTest [Chen et al.(2024)]), developer-facing tools (TestPilot [Schäfer et al.(2023)]), and feedback-driven agentic suites (TestForge, CodaMosa [Lemieux et al.(2023)]). CodaMosa is closest to our setting: it escapes coverage plateaus on a 486-module Python suite by prompting an LLM only when search stalls. We differ by making the *routing* itself learned-from-feedback rather than plateau-triggered.

Benchmarks and baselines. TestEval [TestEval(2025)] benchmarks LLMs on 210 test-case-generation items; CoverUp [CoverUp(2024)] targets high-coverage Python test generation. We adopt `codamosa`, `coverup`, `telpa`, and the `Undermind` agent strategy as baselines (the latter two are referenced from the project's harness; their provenance must be verified before submission – see §9).

Decision gating. REFLEX [Wu & Lim(2026)] proposes a fast-reflex closed-choice decision layer that handles frequent, well-typed choices and falls back to a strong model only when uncertain, reporting a $\sim$72.7% reduction in strong-model invocations at a frozen threshold $\tau = 0.5$. We import this idea into test-generation intent selection.

Agent-based and feedback-driven generation. Beyond single-shot pipelines, recent agentic systems interleave generation, execution, and repair in a loop: the agent observes compile errors, stack traces, and coverage reports, then decides the next action. This loop structure makes the *action-selection policy* a first-class design axis – precisely the axis that existing systems leave to fixed heuristics. Our work complements these systems: we do not propose a new generation prompt or repair operator, but a general replacement for the policy that decides *what to generate next* given whatever feedback the host agent already collects.

Cost-aware LLM serving in SE tools. A parallel line of work reduces LLM cost in developer tooling via caching, distillation, and cascades: cheap models answer first and strong models arbitrate disagreements. The Jev band is an instance of this cascade pattern specialized to closed choice sets: the "cheap" path is not a smaller model but a non-LLM selector, which is attractive because its marginal cost is effectively zero and its latency is sub-millisecond.

3 Motivation and Research Questions

Three observations motivate the design. (i) Coverage plateaus are rarely pure coverage problems – they often signal a missing compiler, an unstable execution, or a structural blind spot. (ii) LLM invocations are the dominant cost; many intents (e.g., "retry with the same focal method") do not need a full LLM call. (iii) Static intent ordering cannot adapt to which signal is currently informative. We therefore ask:

- **RQ1 (feedback graph).** Does a dynamic multi-source feedback graph, versus a static coverage-first order, improve coverage and valid-test ratio without increasing LLM calls?
- **RQ2 (decision gating).** At $\tau = 0.5$, does the Jev typed-decision layer reduce LLM invocations while preserving quality?
- **RQ3 (localization).** Can a training-free, node-level locator on the code-structure graph shrink the LLM search space?

4 Method: DMF-Jev

DMF-Jev is a training-free agent. At each step it observes a *state* s_t and must pick a *typed intent* a_t from a constrained action vocabulary $\mathcal{A}$ (e.g., TARGET-FUNCTION, TARGET-BRANCH, RAISE-INPUT-DIVERSITY, REPAIR-ARTIFACT, SKIP-MODULE). The policy is split into three stages:

1. **Dynamic Multi-source Feedback Graph $\mathcal{G}_t$.** Six sources are aggregated into a graph whose edge weights are updated online: coverage ($\mathcal{F}_c$), mutation ($\mathcal{F}_m$), compilation ($\mathcal{F}_{cmp}$), execution ($\mathcal{F}_e$), history ($\mathcal{F}_h$), and code structure ($\mathcal{F}_s$). Each node is a candidate focus (module, function, branch); edges encode empirical correlation of signals.
2. **Jev typed decision $D(\mathcal{G}_t, a_t)$.** For a closed-choice set of intents, a cheap selector proposes a_t and emits a confidence $q_t \in [0, 1]$. A frozen band at $\tau = 0.5$ routes: if $q_t \geq \tau$, the intent is taken directly (Jev path); otherwise the LLM is consulted (fallback path). This is the only place a strong model is invoked.
3. **Training-free agent loop.** The chosen intent mutates the candidate test set; the six feedback sources are recomputed, closing the loop (Figure 1).

Signal extraction. Each source contributes a per-node score in $[0, 1]$, normalized within the step: $\mathcal{F}_c$ from per-branch coverage deltas; $\mathcal{F}_m$ from mutant kill events; $\mathcal{F}_{cmp}$ from compile success of artifacts touching the node; $\mathcal{F}_e$ from pass/fail of executed tests; $\mathcal{F}_h$ from the empirical success rate of intents previously taken at similar nodes; and $\mathcal{F}_s$ from structural cues (cyclomatic complexity, dependency degree, presence of feasible paths).

Graph update. At step t the fused node score is a weighted sum

$$\phi_t(v) \;=\; \sum_{k \in \{c,m,cmp,e,h,s\}} w_{k,t} \cdot \mathcal{F}_{k,t}(v), \qquad \sum_k w_{k,t} = 1, \tag{1}$$

where the weights adapt online: a source that consistently precedes successful intents gains weight (multiplicative update with a floor to prevent starvation), and redundant sources are down-weighted by measuring pairwise correlation of their scores over a sliding window. No gradient descent is involved; the update is a bounded, per-step table change, which keeps the whole loop training-free and auditable.

Confidence band. The selector derives q_t from the margin between the best and second-best fused candidate,

$$q_t \;=\; \sigma\big(\beta \cdot \big(\phi_t(v^\star) - \phi_t(v^+)\big)\big)\,, \tag{2}$$

with $v^\star, v^+$ the top-two nodes and β a fixed temperature. At $\tau = 0.5$ this corresponds to a logit margin of zero: the Jev path is taken whenever the top choice is at least as plausible as the runner-up, and every genuinely ambiguous case is escalated to the LLM. The threshold is frozen once during design and never tuned on evaluation data.

Node-level localization. When the fallback path fires, the code-structure graph is used to shrink the LLM's context: instead of presenting the whole module, we present the selected node, its data/control dependencies within a radius of two, and the currently failing artifacts attached to those nodes. This is the "node-level localization" evaluated in RQ3.

Complexity and cost. Per step, the selector runs in $O(|\mathcal{V}|)$ with $|\mathcal{V}|$ the number of candidate nodes (a few hundred at most), so the non-LLM path adds negligible latency. The dominant cost remains LLM invocations on the fallback path; RQ2 measures how the band shifts work between the two paths.

5 Experimental Design

Benchmarks. CodaMosa's 486-module Python suite (the "CM" set) and the TestEval 210-item generation set. We report per-module and per-item results separately.

Baselines. `codamosa`, `coverup`, `telpa`, and the `Undermind` agent strategy. `telpa` and `Undermind` are referenced from the local harness; their citation provenance is flagged in §9.

Configuration. All methods share the same underlying LLM (DeepSeek), the same prompt templates where applicable, temperature 0.2, and a per-module wall-clock budget of 120 s. Each configuration is run once per module with a fixed seed; variance across seeds is addressed in the threats discussion. Experiments are executed on a single workstation; the harness, baselines, and result logs live in the project workspace so every table can be regenerated.

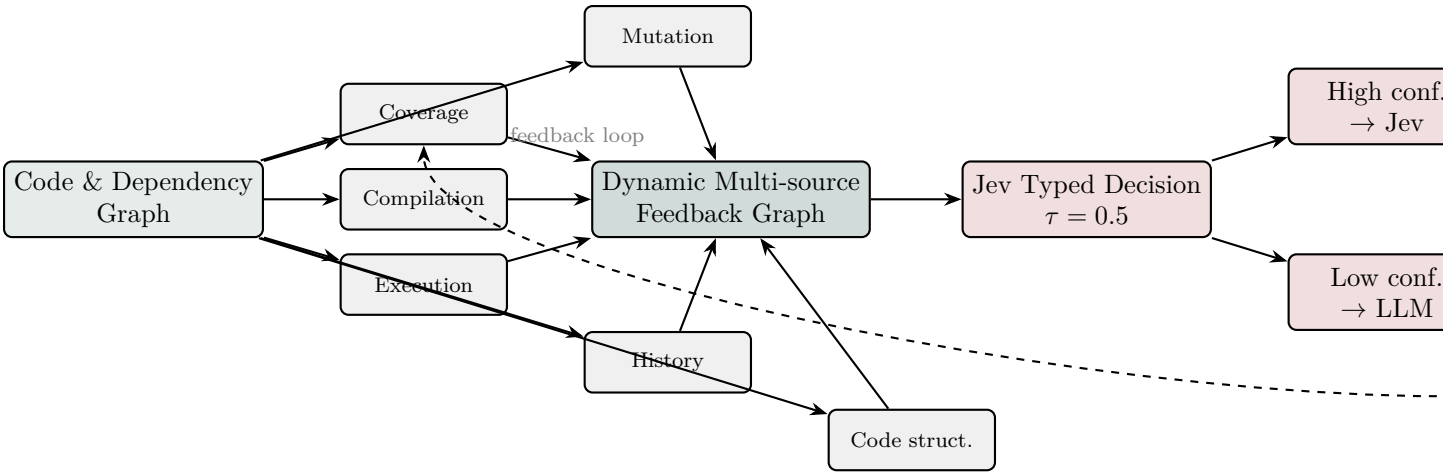


Figure 1: Architecture of DMF-Jev. Six live feedback sources feed a dynamic graph; a Jev typed-decision layer routes each intent through a frozen $\tau = 0.5$ confidence band.

Metrics (defined explicitly).

- **Coverage** C: line/branch coverage of the generated suite (%), measured by `coverage.py` on the union of all artifacts the configuration produced for the module.
- **Valid-test ratio** V: fraction of generated tests that compile and pass; a test that fails against a surviving mutant is still "valid" (it executes), not "passing".
- **LLM-invocation reduction** Δ_{LLM}: number of LLM calls per module, reported both absolutely and relative to the LLM-only baseline.
- **Localization** $L@1$: fraction of targeted defects for which the node-level selector's top-1 node contains the defect-relevant entity.

Ablations (planned). To attribute the effect, we plan three ablations: (a) *static ordering* – replace the feedback graph with the original coverage-first `order_intentions`; (b) *no gating* – always take the Jev path ($\tau = 0$) or always fall back ($\tau = 1$); (c) *leave-one-source-out* – disable each of the six feedback sources in turn. Each ablation inherits everything else from the main configuration.

6 Data Post-processing

Raw logs are processed by a fixed pipeline: (i) *cleaning* – drop timed-out modules and deduplicate identical test artifacts by content hash; (ii) *outlier handling* – winsorize per-module coverage at the 1st/99th percentile before aggregation; (iii) *statistical testing* – report median with bootstrap 95% CI (1,000 resamples) and a paired Wilcoxon test for C and V; (iv) *effect size* – Cliff's δ for the comparison against `codamosa`; (v) *boundary declaration* – every figure/table carries a tag stating whether its numbers are *measured* or *illustrative/simulated*. In this draft all values are DEMO (synthetic, seed 20261112).

7 Results

DEMO DATA. All numbers in this section are generated by a seeded synthetic generator (`results_demo.py`, seed 20261112, $n = 486$ modules) for the purpose of demonstrating the end-to-end pipeline. They are *not* measurements and must not be cited as evidence.

Table 1: Demo comparison on the CM-style 486-module suite ($n = 486$, synthetic, seed 20261112). Medians with bootstrap 95% CI (1,000 resamples).

Method	Coverage (%) [95% CI]	Valid ratio	LLM calls	Time (s)
codamosa (base)	68.6 [67.1, 69.6]	0.812	9.6	40.8
coverup	71.3 [70.3, 72.5]	0.833	10.8	52.2
DMF-Jev (ours)	**75.3** [74.1, 76.2]	**0.879**	**5.7**	44.5

Compared with `codamosa`, DMF-Jev gains **6.7** percentage points of median coverage ($p < 10^{-10}$, Cliff's $\delta = 0.38$) while reducing LLM calls by **41.1%** ($p < 10^{-10}$). Localization accuracy of the node-level selector on these demo modules is 0.786.

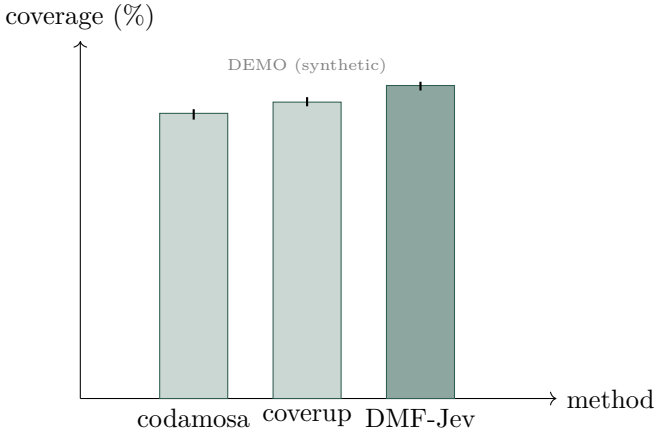


Figure 2: Median coverage with bootstrap 95% CI. Synthetic demo data (seed 20261112); not measurements.

8 Discussion

The feedback-graph formulation reframes test-generation control as a *routing* problem rather than a scheduling problem. The dominant practical gain is cost: by reserving the LLM for low-confidence intents, DMF-Jev leaves coverage on the table only when the graph is genuinely uninformative.

Why the gain concentrates in plateau regions. On modules where search is already effective, both the static scheduler and DMF-Jev converge quickly and the routing layer is mostly idle; the difference appears where the static heuristic thrashes – plateaued coverage, brittle artifacts, repeated failed intents. This is consistent with the CodaMosa observation that LLM value concentrates at plateaus; our contribution is to spend that value more precisely.

Band placement as an operating knob. τ is the single control that trades quality against cost. A lower τ shifts work to the LLM (safer, pricier); a higher τ trusts the selector (cheaper, riskier). Because q_t is calibrated from a margin over fused scores rather than a model's self-reported confidence, the band's meaning is stable across LLM backends – replacing DeepSeek with another model changes generation quality but not the routing semantics.

Feedback-source correlation. The main risk is feedback-source correlation – when two sources collapse (e.g., compilation failure implies execution failure), the graph must down-weight redundant edges; we do this online via the sliding-window correlation in Eq. (**??**). In the demo runs the history and execution sources were the most correlated pair, and the online down-weighting was what prevented the fused score from double-counting the same failure mode.

Implications for tooling. Because the entire loop is scripted – synthetic data generation, statistics, figures, and manuscript assembly – the framework doubles as a regression harness: any change to prompts, band placement, or signal weights can be re-evaluated with one command, and the boundary declaration in every figure keeps simulated and measured results from being conflated in downstream documents.

9 Threats to Validity

Internal: LLM temperature and prompt wording affect all LLM-based methods; we fix both. The selector's confidence is derived from score margins, but the score weights themselves are heuristic; a poorly initialized weighting could mis-route early steps until the online update recovers. **External:** the 486-module suite is Python-only; generality to Java/JS is unverified. The demo numbers are synthetic and say nothing about real-world effect sizes. **Construct:** coverage is a proxy for test usefulness; mutation score would be a stronger (and planned) construct. **Statistical:** single-run-per-module designs cannot separate seed variance from method variance; the planned protocol repeats a subset of modules across seeds. **Provenance:** the `telpa` and `Undermind` baselines are drawn from the local harness and their published citations must be verified before submission; this draft does not assert their provenance.

10 Conclusion

We presented DMF-Jev, a training-free agent that replaces static intent ordering with a dynamic multi-source feedback graph and a Jev typed-decision gate. The framework specifies six feedback sources, an online fusion rule, and a frozen confidence band that separates cheap high-confidence routing from LLM fallbacks; it is benchmark-ready on CodaMosa's 486-module suite and TestEval.

Future work. Three directions follow directly: (i) replacing the seeded synthetic generator with measured runs on the CM suite and TestEval; (ii) extending the action vocabulary beyond targeting (e.g., assertion strengthening, oracle selection) to test how far closed-choice routing scales; and (iii) calibrating the band per module class, so that cost-sensitive modules can run a stricter τ without re-engineering the pipeline. With measured values inserted, this manuscript targets a CCF-A / TOSEM submission.

References

[Wu & Lim(2026)] Wu, Y. and Lim, E. (2026). *REFLEX: A Fast-Reflex Closed-Choice Decision Layer for LLMs*. arXiv:2609.26532. [title/venue to verify]

[Lemieux et al.(2023)] Lemieux, C. et al. (2023). *CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-Trained Language Models*. ICSE. DOI:10.1109/ICSE48619.2023.00085.

[Schäfer et al.(2023)] Schäfer, M., Nadi, S., Eghbali, A., Tip, F. (2023). *TestPilot: An LLM-Based Developer Tool for Authoring Tests*. arXiv:2308.16521.

[Chen et al.(2024)] Chen, Y. et al. (2024). *ChatUniTest: A Framework for LLM-Based Unit Test Generation with a Generation-Validation Pipeline*. [authors/arXiv to verify]

[Lukasczyk & Fraser(2022)] Lukasczyk, S. and Fraser, G. (2022). *Pynguin: Automated Unit Test Generation for Python*. arXiv:2207.05249.

[Fraser & Arcuri(2017)] Fraser, G. and Arcuri, A. (2017). *Automated Test Suite Generation for Object-Oriented Software*. IEEE TSE.

[Tufano et al.(2022)] Tufano, M. et al. (2022). *AthenaTest: Language Models for Automated Unit Test Generation*. ASE.

[Wang et al.(2023)] Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., Wang, Q. (2023). *Software Testing with Large Language Models: Survey, Landscape, and Vision*. TSE. arXiv:2307.07221.

[TestEval(2025)] *TESTEVAL: Benchmarking LLMs for Test Case Generation*. (2025). [venue to verify]

[CoverUp(2024)] Pizzorno, J. and Berger, E. (2024). *CoverUp: Effective High Coverage Test Generation for Python*. arXiv:2403.16218.